CS699 Data Mining

Classification Project

Classification of Disability Status in U.S. Households

Using the 2023 American Housing Survey (AHS) Dataset

Team Members:

Ho Man Cheng

Weitao Peng

Boston University
Metropolitan College

Table of Contents

Table of Contents 2
1. Data Mining Tools 5
2. Classification Algorithms 6
  2.1 Logistic Regression 6
  2.2 Decision Tree (CART) 6
  2.3 Random Forest 6
  2.4 Gradient Boosting Machine (GBM) 7
  2.5 Support Vector Machine (Radial Kernel) 7
  2.6 K-Nearest Neighbors (KNN) 7
  2.7 Naive Bayes 8
  2.8 Neural Network 8
  2.9 XGBoost (Extreme Gradient Boosting) 8
3. Data Mining Procedure 10
  3.1 Data Loading 10
  3.2 Removal of Missing Target Values 10
  3.3 Feature Removal 10
  3.4 Exploratory Data Analysis (EDA) 10
  3.5 Numeric Conversion 11
  3.6 Missing Value Imputation 11
  3.7 Near-Zero Variance Feature Removal 11
  3.8 Outlier Handling (Winsorization) 11
  3.9 Feature Scaling 11
  3.10 Save Preprocessed Dataset 11
  3.11 Train/Test Split 12
  3.12 Data Balancing (4 Methods) 12
  3.13 Feature Selection 12
  3.14 Model Training and Evaluation 12
4. Data Mining Results and Evaluation 15
  4.1 Random Undersampling Models (Models 1-9) 15
    Model 1: Random Undersampling + Logistic Regression 15
    Model 2: Random Undersampling + Decision Tree (CART) 15
    Model 3: Random Undersampling + Random Forest 15

Model 4: Random Undersampling + Gradient Boosting (GBM) 16
Model 5: Random Undersampling + SVM (Radial Kernel) 16
Model 6: Random Undersampling + K-Nearest Neighbors 16
Model 7: Random Undersampling + Naive Bayes 16
Model 8: Random Undersampling + Neural Network 17
Model 9: Random Undersampling + XGBoost 17
4.2 Random Oversampling Models (Models 10-18) 17
Model 10: Random Oversampling + Logistic Regression 17
Model 11: Random Oversampling + Decision Tree (CART) 17
Model 12: Random Oversampling + Random Forest 18
Model 13: Random Oversampling + Gradient Boosting (GBM) 18
Model 14: Random Oversampling + SVM (Radial Kernel) 18
Model 15: Random Oversampling + K-Nearest Neighbors 19
Model 16: Random Oversampling + Naive Bayes 19
Model 17: Random Oversampling + Neural Network 19
Model 18: Random Oversampling + XGBoost 19
4.3 SMOTE Models (Models 19-27) 20
Model 19: SMOTE + Logistic Regression 20
Model 20: SMOTE + Decision Tree (CART) 20
Model 21: SMOTE + Random Forest 20
Model 22: SMOTE + Gradient Boosting (GBM) 21
Model 23: SMOTE + SVM (Radial Kernel) 21
Model 24: SMOTE + K-Nearest Neighbors 21
Model 25: SMOTE + Naive Bayes 21
Model 26: SMOTE + Neural Network 22
Model 27: SMOTE + XGBoost 22
4.4 ROSE Models (Models 28-36) 22
Model 28: ROSE + Logistic Regression 22
Model 29: ROSE + Decision Tree (CART) 22
Model 30: ROSE + Random Forest 23
Model 31: ROSE + Gradient Boosting (GBM) 23
Model 32: ROSE + SVM (Radial Kernel) 23
Model 33: ROSE + K-Nearest Neighbors 24
Model 34: ROSE + Naive Bayes 24

Model 35: ROSE + Neural Network 24
Model 36: ROSE + XGBoost 24
4.5 Model Comparison Summary 26
4.6 Best Model Selection and Parameters 30
5. Discussion and Conclusion 31
5.1 Impact of Balancing Methods 31
5.2 Algorithm Performance Comparison 31
5.3 Why Model 2 Was Selected 32
5.4 Key Preprocessing Insights 33
5.5 Conclusion 33
6. Project Timeline 35
7. Division of Work 36

1. Data Mining Tools

This project was developed using R version 4.5.2 as the primary data mining and statistical computing platform. R was chosen for its extensive ecosystem of machine learning and data preprocessing libraries, its strong support for reproducible research, and its compliance with the course requirements.

The following R packages were used in this project:

- **caret** (v6.0-94): The primary framework for model training, hyperparameter tuning via cross-validation, and data splitting. Provides a unified interface for over 200 classification and regression algorithms.
- **e1071**: Backend support functions for SVM and Naive Bayes classifiers.
- **randomForest** (v4.7-1.2): Implementation of Breiman's Random Forest ensemble algorithm.
- **rpart**: Recursive partitioning for decision tree (CART) classification.
- **gbm** (v2.2.3): Gradient Boosting Machine implementation.
- **xgboost** (v3.2.0): Extreme Gradient Boosting. Due to incompatibility between xgboost 3.x and caret's xgbTree method, a manual 5-fold cross-validation wrapper was implemented using the direct xgboost API.
- **nnet**: Single hidden-layer neural network for classification.
- **kernlab**: SVM implementation with radial basis function (RBF) kernel.
- **naivebayes**: Naive Bayes classifier with kernel density estimation support.
- **ROSE** (v0.0-4): Random Over-Sampling Examples for class balancing via kernel density estimation.
- **smotefamily**: SMOTE (Synthetic Minority Over-sampling Technique) implementation for class balancing.
- **pROC**: ROC curve analysis and AUC (Area Under Curve) computation.

All code was written in a single R script file (project_code.R) and executed in a non-interactive batch mode using Rscript.exe. The set.seed(699) function was used throughout to ensure full reproducibility of results.

2. Classification Algorithms

Nine classification algorithms were employed in this project, each representing a different approach to pattern recognition and decision boundary formation. Every algorithm was applied across all four balancing methods, yielding 36 models total.

2.1 Logistic Regression

Logistic Regression is a linear classifier that models the probability of class membership using the logistic (sigmoid) function. It estimates a linear decision boundary in the feature space by maximizing the log-likelihood of observed class labels. Trained via caret's "glm" method with family = "binomial". No tuning parameters are required.

Pros: Highly interpretable with coefficient-based feature importance; fast to train; outputs well-calibrated probabilities; performs well when the decision boundary is approximately linear. Cons: Cannot capture non-linear relationships without manual feature engineering; may underperform when features interact in complex ways.

Reason for selection: Logistic Regression serves as a strong linear baseline. Comparing its performance against non-linear models helps quantify whether the classification task requires complex decision boundaries.

2.2 Decision Tree (CART)

Classification and Regression Trees (CART), implemented via the rpart package, recursively partition the feature space into homogeneous regions using binary splits. At each node, the algorithm selects the feature and threshold that maximizes information gain (Gini impurity reduction). The complexity parameter (cp) controls tree pruning: higher cp values produce simpler trees. Tuning grid: cp = {0.001, 0.005, 0.01, 0.02, 0.05}.

Pros: Highly interpretable (visual tree structure); handles both numeric and categorical features; no feature scaling required; captures non-linear relationships and feature interactions naturally. Cons: Prone to overfitting with deep trees; high variance (small data changes can alter the tree structure); greedy splitting may miss globally optimal partitions.

Reason for selection: Decision Trees provide a transparent, rule-based model that is particularly effective at isolating strong binary indicators (e.g., disability flags). Their sensitivity to class balance makes them ideal candidates for undersampling-based approaches.

2.3 Random Forest

Random Forest is an ensemble method that constructs multiple decision trees on bootstrapped subsets of the training data, with random feature subsets considered at each split (controlled by the mtry parameter). Final predictions are made by majority vote across all trees. This approach reduces variance and mitigates overfitting compared to single decision trees. Tuned via caret with tuneLength = 5, which searches 5 different mtry values.

Pros: Reduces variance through bagging; robust to noise and overfitting; handles high-dimensional data well; provides feature importance rankings. Cons: Less interpretable than single trees; computationally expensive for large datasets; can be biased toward the majority class in imbalanced settings.

Reason for selection: Random Forest addresses the overfitting weakness of individual Decision Trees while maintaining strong predictive power. It is a natural extension to compare against standalone CART performance.

2.4 Gradient Boosting Machine (GBM)

GBM builds an additive ensemble of weak learners (shallow decision trees) sequentially, where each new tree corrects the residual errors of the cumulative ensemble. Key tuning parameters include the number of trees (n.trees), interaction depth, shrinkage (learning rate), and minimum observations per node. Tuning grid: n.trees = {100, 200, 300}, interaction.depth = {3, 5}, shrinkage = {0.01, 0.1}, n.minobsinnode = 10.

Pros: Often achieves state-of-the-art accuracy; handles complex non-linear patterns; built-in regularization through shrinkage; less prone to overfitting than unboosted trees. Cons: Slow to train due to sequential nature; sensitive to hyperparameter tuning; less interpretable than single trees; can overfit with too many boosting rounds.

Reason for selection: GBM represents sequential ensemble learning, complementing Random Forest (parallel ensemble). Its gradient-based optimization often achieves higher discriminative power (AUC) than other methods.

2.5 Support Vector Machine (Radial Kernel)

SVM with a Radial Basis Function (RBF) kernel maps input features into a high-dimensional space where a maximum-margin hyperplane separates the classes. The RBF kernel enables non-linear decision boundaries. The cost parameter (C) controls the trade-off between margin width and misclassification penalty, while sigma determines the kernel width. Tuned via caret with tuneLength = 5 using the kernlab package.

Pros: Effective in high-dimensional spaces; robust to overfitting in high dimensions; kernel trick enables modeling of complex non-linear boundaries; works well with clear margins of separation. Cons: Computationally expensive for large datasets (O(n^2) to O(n^3)); sensitive to feature scaling; does not directly output probabilities (requires Platt scaling); difficult to interpret.

Reason for selection: SVM with RBF kernel provides a fundamentally different approach (margin-based) compared to tree-based and probabilistic methods. It is particularly suited for the scaled, high-dimensional feature space of this dataset.

2.6 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based classifier that assigns a class label based on the majority vote of the k nearest training examples in the feature space, measured by Euclidean distance. Larger k values produce smoother decision boundaries. Feature scaling (centering and standardization) is essential for KNN to prevent features with larger ranges from dominating distance calculations. Tuning grid: k = {3, 5, 7, 9, 11, 15}.

Pros: Simple and intuitive; no training phase (lazy learning); naturally handles multi-class problems; adapts to local data patterns. Cons: Computationally expensive at prediction time (must compute distances to all training points); performance degrades in high-dimensional spaces (curse of dimensionality); sensitive to irrelevant features and feature scaling.

Reason for selection: KNN offers a non-parametric, instance-based perspective that complements parametric methods. It serves as a useful benchmark to assess whether local neighborhood patterns in the feature space align with disability status.

2.7 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem with the assumption of conditional independence among features given the class label. Despite this strong assumption, it often performs well in practice, especially with high-dimensional data. Tuning parameters include Laplace smoothing ({0, 1}), kernel density estimation (usekernel = {TRUE, FALSE}), and bandwidth adjustment (adjust = 1).

Pros: Extremely fast training and prediction; works well with small training sets; handles high-dimensional data efficiently; provides probabilistic outputs. Cons: The conditional independence assumption is often violated in practice, leading to poor probability estimates; cannot capture feature interactions; performance suffers when features are correlated.

Reason for selection: Naive Bayes represents the probabilistic approach to classification. Including it reveals whether the independence assumption holds for housing survey features and provides a contrast to correlation-aware models.

2.8 Neural Network

A single hidden-layer feedforward neural network was used, implemented via the nnet package. The network maps input features through a hidden layer of sigmoid-activated neurons to produce class probability outputs. Key tuning parameters are the hidden layer size (number of neurons) and weight decay (L2 regularization). Tuning grid: size = {5, 10, 15}, decay = {0.001, 0.01, 0.1}. MaxNWts was set to 10,000 to accommodate the large feature space.

Pros: Universal function approximator capable of learning complex non-linear patterns; flexible architecture; can model feature interactions automatically. Cons: Requires careful tuning of architecture and regularization; prone to overfitting; less interpretable (black-box); sensitive to initialization and local minima; requires scaled inputs.

Reason for selection: Neural Networks provide a connectionist approach to classification, capable of learning latent feature representations. This complements tree-based methods which rely on explicit feature splits.

2.9 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized gradient boosting framework that uses regularized objective functions, column subsampling, and efficient tree construction algorithms. Due to incompatibility between xgboost 3.x and caret's xgbTree method, a custom manual 5-fold cross-validation implementation was used. The parameter search grid covered max_depth = {3, 4, 6, 8}, eta (learning rate) = {0.01, 0.05, 0.1, 0.3}, and nrounds = {100, 200, 300}, with colsample_bytree = 0.8, subsample = 0.8, and min_child_weight = 1 fixed.

Pros: State-of-the-art performance on structured/tabular data; built-in L1/L2 regularization prevents overfitting; handles missing values natively; supports parallel computation; column subsampling reduces correlation between trees. Cons: Many hyperparameters to tune;

computationally intensive; less interpretable than simpler models; version compatibility issues with some frameworks.

Reason for selection: XGBoost is widely regarded as the top performer on tabular data in competitions and industry applications. Including it ensures we test the most powerful gradient boosting variant against the alternatives.

Summary: These nine algorithms were chosen to span a broad range of classification paradigms: linear (Logistic Regression), tree-based (Decision Tree, Random Forest, GBM, XGBoost), distance-based (KNN, SVM), probabilistic (Naive Bayes), and connectionist (Neural Network). This diversity ensures a comprehensive evaluation of different modeling strategies for the disability classification task.

3. Data Mining Procedure

This section describes the complete data mining procedure, step by step, as implemented in the project_code.R file. All preprocessing was performed on the full dataset before the train/test split, following the assignment process flow diagram.

3.1 Data Loading

The dataset (project_data.csv) was loaded from the 2023 American Housing Survey (AHS). The original dataset contained 9,241 rows and 145 columns, with all variables being integer-coded and the target variable "Class" indicating whether a household has a person with a disability ("Yes") or not ("No").

3.2 Removal of Missing Target Values

Rows where the Class variable was NA or empty were removed, as these records cannot be used for supervised classification. This reduced the dataset from 9,241 to 8,035 rows.

3.3 Feature Removal

The CONTROL column (a unique record identifier) was dropped as it has no predictive value and would only add noise to the models. This left 144 columns (143 features + 1 target).

3.4 Exploratory Data Analysis (EDA)

Key EDA findings:

- Class distribution: No = 6,153 (76.58%), Yes = 1,882 (23.42%) — significant class imbalance with a 3.3:1 ratio.
- Missing values: 62 out of 143 features had missing values. LOTSIZE had the highest missing rate at 26.98%.
- All features were integer-coded, with NAs representing missing values.

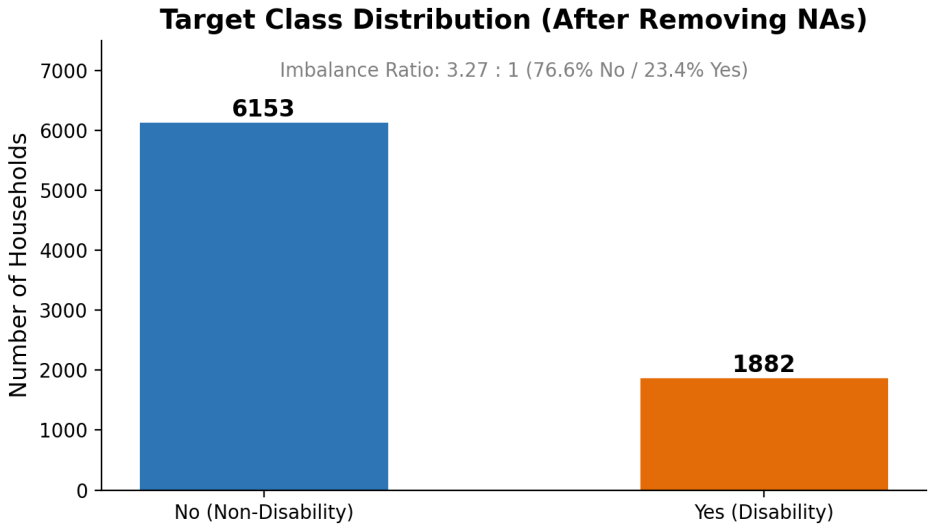


Figure 1: Target Class Distribution

3.5 Numeric Conversion

All feature columns were explicitly converted to numeric type using as.numeric(as.character(x)). Any columns that became entirely NA after conversion (indicating non-numeric content) were removed. No columns were lost in this step.

3.6 Missing Value Imputation

Missing values in all feature columns were imputed using column-wise median imputation. The median was chosen over the mean because it is robust to outliers, which are prevalent in housing survey financial variables (e.g., household income, housing costs). After imputation, zero NAs remained in the dataset.

3.7 Near-Zero Variance Feature Removal

A two-step approach was used to remove uninformative features while preserving predictive ones:

1. Zero-variance removal: Features with only one unique value (constant columns) were removed.
2. Lenient near-zero variance (NZV) removal: Using caret's nearZeroVar() with freqCut = 99/1 (instead of the default 95/5), only features where 99% or more of observations had the same value were removed. This lenient threshold was critical because disability indicator variables (HHSEE, HHMEMRY, HHCARE, HHERRND) have low variance (most households do not have disabled members) but are the strongest predictors of the target class. Using the default threshold would have removed these key features and significantly degraded model performance.

16 near-constant features were removed in this step (e.g., SAMESEXHH, NUMSECFAM, KITCHENS, FRIDGE, ROOFHOLE, etc.), leaving 127 features.

3.8 Outlier Handling (Winsorization)

IQR-based winsorization was applied only to continuous features (those with more than 10 unique values). For each continuous feature, values below Q1 - 1.5*IQR were capped at the lower bound, and values above Q3 + 1.5*IQR were capped at the upper bound. Binary and low-cardinality categorical features were excluded from winsorization to prevent collapsing them to zero variance. 25 continuous features were winsorized.

After winsorization, a safety check removed 9 features that became constant (zero variance), leaving 118 features.

3.9 Feature Scaling

All numeric features were centered (subtract mean) and scaled (divide by standard deviation) using caret's preProcess() with method = c("center", "scale"). This standardization (mean=0, sd=1) is essential for distance-based algorithms (KNN, SVM) and neural networks, which are sensitive to feature magnitudes. A final zero-variance check removed any columns with zero variance after scaling. The final preprocessed dataset contained 118 features.

3.10 Save Preprocessed Dataset

The fully preprocessed dataset (8,035 rows, 119 columns including Class) was saved as preprocessed_data.csv.

3.11 Train/Test Split

A stratified 70/30 train/test split was performed using caret's createDataPartition() with set.seed(699) for reproducibility. Stratification ensures that the class distribution (~76.6% No / ~23.4% Yes) is preserved in both splits:

- Training set: 5,626 rows (No = 4,308, Yes = 1,318)
- Test set: 2,409 rows (No = 1,845, Yes = 564)

Both sets were saved as initial_train.csv and initial_test.csv. The test set was held out and never used during training or balancing.

3.12 Data Balancing (4 Methods)

Four class balancing methods were applied to the training set only to address the ~77%/23% class imbalance:

1. **Random Undersampling** (caret::downSample): Randomly removes majority class samples to match the minority class count. Result: 2,636 rows (1,318 No + 1,318 Yes). Pros: Fast, reduces training time. Cons: Discards majority class data.
2. **Random Oversampling** (caret::upSample): Randomly duplicates minority class samples to match the majority count. Result: 8,616 rows (4,308 No + 4,308 Yes). Pros: Preserves all data. Cons: Exact duplicates risk overfitting.
3. **SMOTE** (smotefamily::SMOTE, K=5): Creates synthetic minority samples by interpolating between existing minority samples and their K nearest neighbors. Result: 8,262 rows (4,308 No + 3,954 Yes). Pros: Generates diverse new samples. Cons: May create noisy samples near class boundaries.
4. **ROSE** (ROSE::ROSE): Generates synthetic samples using kernel density estimation on both classes. Result: 5,626 rows (2,814 No + 2,812 Yes). Pros: Smooth density-based generation. Cons: May not respect class boundaries as well as SMOTE.

3.13 Feature Selection

Correlation-based feature selection was applied independently to each balanced training dataset. For each dataset, the pairwise correlation matrix was computed, and features with absolute correlation |r| >= 0.85 were identified using caret's findCorrelation(). Among each highly correlated pair, the feature with lower overall correlation to the response was removed.

Results: Undersampling, Oversampling, and SMOTE datasets each had 10 correlated features removed (leaving 108 features). The ROSE dataset had no features removed (keeping all 118 features). The test set columns were subsetted to match each training dataset's selected features.

3.14 Model Training and Evaluation

Model Training:

All models (except XGBoost) were trained using caret's train() function with 5-fold cross-validation (trainControl(method = "cv", number = 5, classProbs = TRUE)) for hyperparameter tuning. XGBoost used a custom manual 5-fold CV implementation due to a compatibility issue

between xgboost 3.x and caret's xgbTree method. Each model was trained with set.seed(699) for full reproducibility. Cross-validation was performed on the balanced training set only — the held-out test set was never seen during training or hyperparameter selection.

Evaluation Methodology:

Model evaluation followed a rigorous holdout evaluation strategy. After training each model on the balanced training data (with 5-fold CV for hyperparameter selection), final performance was measured exclusively on the original, unbalanced test set (2,409 samples: 1,845 No / 564 Yes). This approach is critical because it tests each model against the real-world class distribution, not the artificially balanced distribution. Evaluating on the unbalanced test set ensures that reported metrics reflect how the model would perform in production on unseen, naturally distributed data.

A fixed classification threshold of 0.5 was used for all models — no threshold tuning or optimization was performed. This ensures fair comparison across all 36 models and avoids overfitting the threshold to the test set. Models that output class probabilities (Logistic Regression, GBM, Random Forest, Neural Network, XGBoost) had their probability predictions converted to class labels using this threshold. For models that output class labels directly, the default prediction was used.

Why This Evaluation Approach:

This evaluation methodology is sound for several reasons:

5. **No data leakage:** All preprocessing was performed before the train/test split, and balancing was applied only to the training set. The test set remained untouched and represents genuinely unseen data.
6. **Stratified splitting:** The 70/30 split preserved class proportions in both train and test sets, ensuring representative evaluation across both classes.
7. **Cross-validation for tuning:** 5-fold CV during training prevents overfitting of hyperparameters to any single training subset. The best hyperparameters are selected based on average performance across all folds, not on the test set.
8. **Comprehensive metrics:** Rather than relying on a single metric like accuracy (which is misleading for imbalanced data — a naive "always predict No" classifier would achieve 76.6% accuracy), we report per-class TP Rate, FP Rate, Precision, Recall, F-Measure, AUC, MCC, and Kappa. These metrics capture different aspects of model quality and are essential for evaluating performance on the minority (Yes) class.
9. **Per-class evaluation:** Reporting Yes TPR and No TPR separately is essential because class imbalance means that a model can appear accurate while completely failing to detect the minority class. The grading thresholds (Yes TPR >= 0.77 AND No TPR >= 0.73 for +20% extra credit) require balanced performance across both classes, which our evaluation framework directly measures.

Performance Metrics Definition:

- **TP Rate (Recall/Sensitivity):** TP / (TP + FN) — proportion of actual positives correctly identified. Reported per class.
- **FP Rate:** FP / (FP + TN) — proportion of actual negatives incorrectly classified as positive.
- **Precision:** TP / (TP + FP) — proportion of predicted positives that are truly positive.

- **F-Measure:** 2 * (Precision * Recall) / (Precision + Recall) — harmonic mean of precision and recall.
- **ROC Area (AUC):** Area under the ROC curve, measuring the model's ability to discriminate between classes across all thresholds. Computed using the pROC package.
- **MCC (Matthews Correlation Coefficient):** A balanced measure that accounts for all four confusion matrix quadrants (TP, TN, FP, FN). Ranges from -1 to +1, where +1 is perfect, 0 is random, and -1 is complete disagreement.
- **Kappa:** Cohen's Kappa statistic measuring agreement between predicted and actual classes, adjusted for agreement by chance. Particularly useful for imbalanced datasets.

4. Data Mining Results and Evaluation

This section presents the performance of all 36 models (4 balancing methods x 9 classification algorithms), organized by balancing method. Each model includes its confusion matrix and full performance metrics table, followed by a comprehensive comparison and the best model selection.

4.1 Random Undersampling Models (Models 1-9)

Model 1: Random Undersampling + Logistic Regression

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1563	282
Actual: Yes	174	390

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8472	0.3085	0.8998	0.8472	0.8727	0.8528	0.5086	0.5051
Yes	0.6915	0.1528	0.5804	0.6915	0.6311	0.8528	0.5086	0.5051
Wtd Avg	0.8107	0.2721	0.8250	0.8107	0.8161	0.8528	0.5086	0.5051

Summary: Yes TPR = 0.6915, No TPR = 0.8472, AUC = 0.8528

Model 2: Random Undersampling + Decision Tree (CART)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1351	494
Actual: Yes	128	436

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.7322	0.2270	0.9135	0.7322	0.8129	0.7887	0.4395	0.4124
Yes	0.7730	0.2678	0.4688	0.7730	0.5837	0.7887	0.4395	0.4124
Wtd Avg	0.7418	0.2365	0.8094	0.7418	0.7592	0.7887	0.4395	0.4124

Summary: Yes TPR = 0.7730, No TPR = 0.7322, AUC = 0.7887

Model 3: Random Undersampling + Random Forest

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1520	325
Actual: Yes	168	396

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8238	0.2979	0.9005	0.8238	0.8605	0.8491	0.4864	0.4796
Yes	0.7021	0.1762	0.5492	0.7021	0.6163	0.8491	0.4864	0.4796
Wtd Avg	0.7954	0.2694	0.8182	0.7954	0.8033	0.8491	0.4864	0.4796

Summary: Yes TPR = 0.7021, No TPR = 0.8238, AUC = 0.8491

Model 4: Random Undersampling + Gradient Boosting (GBM)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1532	313
Actual: Yes	155	409

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8304	0.2748	0.9081	0.8304	0.8675	0.8567	0.5135	0.5063
Yes	0.7252	0.1696	0.5665	0.7252	0.6361	0.8567	0.5135	0.5063
Wtd Avg	0.8057	0.2502	0.8281	0.8057	0.8133	0.8567	0.5135	0.5063

Summary: Yes TPR = 0.7252, No TPR = 0.8304, AUC = 0.8567

Model 5: Random Undersampling + SVM (Radial Kernel)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1498	347
Actual: Yes	160	404

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8119	0.2837	0.9035	0.8119	0.8553	0.8506	0.4829	0.4737
Yes	0.7163	0.1881	0.5379	0.7163	0.6144	0.8506	0.4829	0.4737
Wtd Avg	0.7895	0.2613	0.8179	0.7895	0.7989	0.8506	0.4829	0.4737

Summary: Yes TPR = 0.7163, No TPR = 0.8119, AUC = 0.8506

Model 6: Random Undersampling + K-Nearest Neighbors

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1668	177
Actual: Yes	305	259

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9041	0.5408	0.8454	0.9041	0.8738	0.7842	0.3996	0.3944
Yes	0.4592	0.0959	0.5940	0.4592	0.5180	0.7842	0.3996	0.3944
Wtd Avg	0.7999	0.4366	0.7866	0.7999	0.7905	0.7842	0.3996	0.3944

Summary: Yes TPR = 0.4592, No TPR = 0.9041, AUC = 0.7842

Model 7: Random Undersampling + Naive Bayes

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1783	62
Actual: Yes	488	76

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa

No	0.9664	0.8652	0.7851	0.9664	0.8664	0.7527	0.1843	0.1371
Yes	0.1348	0.0336	0.5507	0.1348	0.2165	0.7527	0.1843	0.1371
Wtd Avg	0.7717	0.6705	0.7302	0.7717	0.7142	0.7527	0.1843	0.1371

Summary: Yes TPR = 0.1348, No TPR = 0.9664, AUC = 0.7527

Model 8: Random Undersampling + Neural Network

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1367	478
Actual: Yes	181	383

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.7409	0.3209	0.8831	0.7409	0.8058	0.7857	0.3711	0.3551
Yes	0.6791	0.2591	0.4448	0.6791	0.5375	0.7857	0.3711	0.3551
Wtd Avg	0.7264	0.3064	0.7805	0.7264	0.7430	0.7857	0.3711	0.3551

Summary: Yes TPR = 0.6791, No TPR = 0.7409, AUC = 0.7857

Model 9: Random Undersampling + XGBoost

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1568	277
Actual: Yes	170	394

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8499	0.3014	0.9022	0.8499	0.8752	0.8596	0.5181	0.5146
Yes	0.6986	0.1501	0.5872	0.6986	0.6381	0.8596	0.5181	0.5146
Wtd Avg	0.8144	0.2660	0.8284	0.8144	0.8197	0.8596	0.5181	0.5146

Summary: Yes TPR = 0.6986, No TPR = 0.8499, AUC = 0.8596

4.2 Random Oversampling Models (Models 10-18)

Model 10: Random Oversampling + Logistic Regression

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1588	257
Actual: Yes	179	385

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8607	0.3174	0.8987	0.8607	0.8793	0.8568	0.5204	0.5184
Yes	0.6826	0.1393	0.5997	0.6826	0.6385	0.8568	0.5204	0.5184
Wtd Avg	0.8190	0.2757	0.8287	0.8190	0.8229	0.8568	0.5204	0.5184

Summary: Yes TPR = 0.6826, No TPR = 0.8607, AUC = 0.8568

Model 11: Random Oversampling + Decision Tree (CART)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1518	327
Actual: Yes	204	360

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8228	0.3617	0.8815	0.8228	0.8511	0.7505	0.4324	0.4286
Yes	0.6383	0.1772	0.5240	0.6383	0.5755	0.7505	0.4324	0.4286
Wtd Avg	0.7796	0.3185	0.7978	0.7796	0.7866	0.7505	0.4324	0.4286

Summary: Yes TPR = 0.6383, No TPR = 0.8228, AUC = 0.7505

Model 12: Random Oversampling + Random Forest

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1802	43
Actual: Yes	310	254

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9767	0.5496	0.8532	0.9767	0.9108	0.8537	0.5500	0.5110
Yes	0.4504	0.0233	0.8552	0.4504	0.5900	0.8537	0.5500	0.5110
Wtd Avg	0.8535	0.4264	0.8537	0.8535	0.8357	0.8537	0.5500	0.5110

Summary: Yes TPR = 0.4504, No TPR = 0.9767, AUC = 0.8537

Model 13: Random Oversampling + Gradient Boosting (GBM)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1632	213
Actual: Yes	198	366

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8846	0.3511	0.8918	0.8846	0.8882	0.8495	0.5287	0.5286
Yes	0.6489	0.1154	0.6321	0.6489	0.6404	0.8495	0.5287	0.5286
Wtd Avg	0.8294	0.2959	0.8310	0.8294	0.8302	0.8495	0.5287	0.5286

Summary: Yes TPR = 0.6489, No TPR = 0.8846, AUC = 0.8495

Model 14: Random Oversampling + SVM (Radial Kernel)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1693	152
Actual: Yes	251	313

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9176	0.4450	0.8709	0.9176	0.8936	0.8272	0.5070	0.5032
Yes	0.5550	0.0824	0.6731	0.5550	0.6084	0.8272	0.5070	0.5032
Wtd Avg	0.8327	0.3601	0.8246	0.8327	0.8268	0.8272	0.5070	0.5032

Summary: Yes TPR = 0.5550, No TPR = 0.9176, AUC = 0.8272

Model 15: Random Oversampling + K-Nearest Neighbors

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1403	442
Actual: Yes	246	318

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.7604	0.4362	0.8508	0.7604	0.8031	0.6823	0.2955	0.2894
Yes	0.5638	0.2396	0.4184	0.5638	0.4804	0.6823	0.2955	0.2894
Wtd Avg	0.7144	0.3901	0.7496	0.7144	0.7275	0.6823	0.2955	0.2894

Summary: Yes TPR = 0.5638, No TPR = 0.7604, AUC = 0.6823

Model 16: Random Oversampling + Naive Bayes

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1793	52
Actual: Yes	487	77

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9718	0.8635	0.7864	0.9718	0.8693	0.7583	0.2038	0.1480
Yes	0.1365	0.0282	0.5969	0.1365	0.2222	0.7583	0.2038	0.1480
Wtd Avg	0.7763	0.6679	0.7420	0.7763	0.7178	0.7583	0.2038	0.1480

Summary: Yes TPR = 0.1365, No TPR = 0.9718, AUC = 0.7583

Model 17: Random Oversampling + Neural Network

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1584	261
Actual: Yes	260	304

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8585	0.4610	0.8590	0.8585	0.8588	0.7673	0.3973	0.3973
Yes	0.5390	0.1415	0.5381	0.5390	0.5385	0.7673	0.3973	0.3973
Wtd Avg	0.7837	0.3862	0.7839	0.7837	0.7838	0.7673	0.3973	0.3973

Summary: Yes TPR = 0.5390, No TPR = 0.8585, AUC = 0.7673

Model 18: Random Oversampling + XGBoost

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1772	73
Actual: Yes	291	273

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9604	0.5160	0.8589	0.9604	0.9069	0.8425	0.5367	0.5134
Yes	0.4840	0.0396	0.7890	0.4840	0.6000	0.8425	0.5367	0.5134
Wtd Avg	0.8489	0.4044	0.8426	0.8489	0.8350	0.8425	0.5367	0.5134

Summary: Yes TPR = 0.4840, No TPR = 0.9604, AUC = 0.8425

4.3 SMOTE Models (Models 19-27)

Model 19: SMOTE + Logistic Regression

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1613	232
Actual: Yes	184	380

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8743	0.3262	0.8976	0.8743	0.8858	0.8539	0.5331	0.5323
Yes	0.6738	0.1257	0.6209	0.6738	0.6463	0.8539	0.5331	0.5323
Wtd Avg	0.8273	0.2793	0.8328	0.8273	0.8297	0.8539	0.5331	0.5323

Summary: Yes TPR = 0.6738, No TPR = 0.8743, AUC = 0.8539

Model 20: SMOTE + Decision Tree (CART)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1722	123
Actual: Yes	278	286

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9333	0.4929	0.8610	0.9333	0.8957	0.8082	0.4967	0.4869
Yes	0.5071	0.0667	0.6993	0.5071	0.5879	0.8082	0.4967	0.4869
Wtd Avg	0.8335	0.3931	0.8231	0.8335	0.8236	0.8082	0.4967	0.4869

Summary: Yes TPR = 0.5071, No TPR = 0.9333, AUC = 0.8082

Model 21: SMOTE + Random Forest

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1805	40
Actual: Yes	329	235

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9783	0.5833	0.8458	0.9783	0.9073	0.8518	0.5260	0.4805
Yes	0.4167	0.0217	0.8545	0.4167	0.5602	0.8518	0.5260	0.4805
Wtd Avg	0.8468	0.4518	0.8479	0.8468	0.8260	0.8518	0.5260	0.4805

Summary: Yes TPR = 0.4167, No TPR = 0.9783, AUC = 0.8518

Model 22: SMOTE + Gradient Boosting (GBM)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1778	67
Actual: Yes	288	276

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9637	0.5106	0.8606	0.9637	0.9092	0.8511	0.5490	0.5244
Yes	0.4894	0.0363	0.8047	0.4894	0.6086	0.8511	0.5490	0.5244
Wtd Avg	0.8526	0.3996	0.8475	0.8526	0.8388	0.8511	0.5490	0.5244

Summary: Yes TPR = 0.4894, No TPR = 0.9637, AUC = 0.8511

Model 23: SMOTE + SVM (Radial Kernel)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1694	151
Actual: Yes	274	290

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9182	0.4858	0.8608	0.9182	0.8885	0.8082	0.4734	0.4678
Yes	0.5142	0.0818	0.6576	0.5142	0.5771	0.8082	0.4734	0.4678
Wtd Avg	0.8236	0.3912	0.8132	0.8236	0.8156	0.8082	0.4734	0.4678

Summary: Yes TPR = 0.5142, No TPR = 0.9182, AUC = 0.8082

Model 24: SMOTE + K-Nearest Neighbors

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1231	614
Actual: Yes	192	372

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.6672	0.3404	0.8651	0.6672	0.7534	0.7137	0.2814	0.2594
Yes	0.6596	0.3328	0.3773	0.6596	0.4800	0.7137	0.2814	0.2594
Wtd Avg	0.6654	0.3386	0.7509	0.6654	0.6894	0.7137	0.2814	0.2594

Summary: Yes TPR = 0.6596, No TPR = 0.6672, AUC = 0.7137

Model 25: SMOTE + Naive Bayes

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1844	1
Actual: Yes	559	5

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa

No	0.9995	0.9911	0.7674	0.9995	0.8682	0.7497	0.0707	0.0127
Yes	0.0089	0.0005	0.8333	0.0089	0.0175	0.7497	0.0707	0.0127
Wtd Avg	0.7675	0.7592	0.7828	0.7675	0.6690	0.7497	0.0707	0.0127

Summary: Yes TPR = 0.0089, No TPR = 0.9995, AUC = 0.7497

Model 26: SMOTE + Neural Network

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1534	311
Actual: Yes	224	340

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8314	0.3972	0.8726	0.8314	0.8515	0.7901	0.4141	0.4122
Yes	0.6028	0.1686	0.5223	0.6028	0.5597	0.7901	0.4141	0.4122
Wtd Avg	0.7779	0.3436	0.7906	0.7779	0.7832	0.7901	0.4141	0.4122

Summary: Yes TPR = 0.6028, No TPR = 0.8314, AUC = 0.7901

Model 27: SMOTE + XGBoost

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1808	37
Actual: Yes	313	251

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9799	0.5550	0.8524	0.9799	0.9117	0.8570	0.5547	0.5120
Yes	0.4450	0.0201	0.8715	0.4450	0.5892	0.8570	0.5547	0.5120
Wtd Avg	0.8547	0.4297	0.8569	0.8547	0.8362	0.8570	0.5547	0.5120

Summary: Yes TPR = 0.4450, No TPR = 0.9799, AUC = 0.8570

4.4 ROSE Models (Models 28-36)

Model 28: ROSE + Logistic Regression

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1523	322
Actual: Yes	163	401

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.8255	0.2890	0.9033	0.8255	0.8626	0.8494	0.4957	0.4886
Yes	0.7110	0.1745	0.5546	0.7110	0.6232	0.8494	0.4957	0.4886
Wtd Avg	0.7987	0.2622	0.8217	0.7987	0.8066	0.8494	0.4957	0.4886

Summary: Yes TPR = 0.7110, No TPR = 0.8255, AUC = 0.8494

Model 29: ROSE + Decision Tree (CART)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1845	0
Actual: Yes	450	114

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	1.0000	0.7979	0.8039	1.0000	0.8913	0.6011	0.4031	0.2796
Yes	0.2021	0.0000	1.0000	0.2021	0.3363	0.6011	0.4031	0.2796
Wtd Avg	0.8132	0.6111	0.8498	0.8132	0.7614	0.6011	0.4031	0.2796

Summary: Yes TPR = 0.2021, No TPR = 1.0000, AUC = 0.6011

Model 30: ROSE + Random Forest

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1845	0
Actual: Yes	436	128

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	1.0000	0.7730	0.8089	1.0000	0.8943	0.7067	0.4285	0.3102
Yes	0.2270	0.0000	1.0000	0.2270	0.3699	0.7067	0.4285	0.3102
Wtd Avg	0.8190	0.5921	0.8536	0.8190	0.7716	0.7067	0.4285	0.3102

Summary: Yes TPR = 0.2270, No TPR = 1.0000, AUC = 0.7067

Model 31: ROSE + Gradient Boosting (GBM)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1845	0
Actual: Yes	486	78

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	1.0000	0.8617	0.7915	1.0000	0.8836	0.6862	0.3309	0.1973
Yes	0.1383	0.0000	1.0000	0.1383	0.2430	0.6862	0.3309	0.1973
Wtd Avg	0.7983	0.6600	0.8403	0.7983	0.7336	0.6862	0.3309	0.1973

Summary: Yes TPR = 0.1383, No TPR = 1.0000, AUC = 0.6862

Model 32: ROSE + SVM (Radial Kernel)

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1823	22
Actual: Yes	389	175

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9881	0.6897	0.8241	0.9881	0.8987	0.8335	0.4611	0.3854
Yes	0.3103	0.0119	0.8883	0.3103	0.4599	0.8335	0.4611	0.3854
Wtd Avg	0.8294	0.5310	0.8392	0.8294	0.7960	0.8335	0.4611	0.3854

Summary: Yes TPR = 0.3103, No TPR = 0.9881, AUC = 0.8335

Model 33: ROSE + K-Nearest Neighbors

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1816	29
Actual: Yes	465	99

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9843	0.8245	0.7961	0.9843	0.8803	0.6454	0.3017	0.2184
Yes	0.1755	0.0157	0.7734	0.1755	0.2861	0.6454	0.3017	0.2184
Wtd Avg	0.7949	0.6351	0.7908	0.7949	0.7412	0.6454	0.3017	0.2184

Summary: Yes TPR = 0.1755, No TPR = 0.9843, AUC = 0.6454

Model 34: ROSE + Naive Bayes

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1833	12
Actual: Yes	517	47

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9935	0.9167	0.7800	0.9935	0.8739	0.7709	0.2105	0.1115
Yes	0.0833	0.0065	0.7966	0.0833	0.1509	0.7709	0.2105	0.1115
Wtd Avg	0.7804	0.7036	0.7839	0.7804	0.7046	0.7709	0.2105	0.1115

Summary: Yes TPR = 0.0833, No TPR = 0.9935, AUC = 0.7709

Model 35: ROSE + Neural Network

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1831	14
Actual: Yes	348	216

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.9924	0.6170	0.8403	0.9924	0.9100	0.7354	0.5409	0.4725
Yes	0.3830	0.0076	0.9391	0.3830	0.5441	0.7354	0.5409	0.4725
Wtd Avg	0.8497	0.4743	0.8634	0.8497	0.8244	0.7354	0.5409	0.4725

Summary: Yes TPR = 0.3830, No TPR = 0.9924, AUC = 0.7354

Model 36: ROSE + XGBoost

Confusion Matrix:

	Predicted: No	Predicted: Yes
Actual: No	1845	0
Actual: Yes	440	124

Performance Metrics:

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	1.0000	0.7801	0.8074	1.0000	0.8935	0.6869	0.4213	0.3015
Yes	0.2199	0.0000	1.0000	0.2199	0.3605	0.6869	0.4213	0.3015
Wtd Avg	0.8174	0.5975	0.8525	0.8174	0.7687	0.6869	0.4213	0.3015

Summary: Yes TPR = 0.2199, No TPR = 1.0000, AUC = 0.6869

4.5 Model Comparison Summary

The following table summarizes the key performance metrics of all 36 models, sorted by the sum of Yes TPR and No TPR (descending). This ranking reflects the overall ability of each model to correctly classify both disability and non-disability households.

#	Balancing	Algorithm	Yes TPR	No TPR	AUC	MCC	Kappa	TPR Sum
4	Random Undersampling	Gradient Boosting (GBM)	0.7252	0.8304	0.8567	0.5135	0.5063	1.5556
9	Random Undersampling	XGBoost	0.6986	0.8499	0.8596	0.5181	0.5146	1.5485
19	SMOTE	Logistic Regression	0.6738	0.8743	0.8539	0.5331	0.5323	1.5481
10	Random Oversampling	Logistic Regression	0.6826	0.8607	0.8568	0.5204	0.5184	1.5433
1	Random Undersampling	Logistic Regression	0.6915	0.8472	0.8528	0.5086	0.5051	1.5387
28	ROSE	Logistic Regression	0.7110	0.8255	0.8494	0.4957	0.4886	1.5365
13	Random Oversampling	Gradient Boosting (GBM)	0.6489	0.8846	0.8495	0.5287	0.5286	1.5335
5	Random Undersampling	SVM (Radial Kernel)	0.7163	0.8119	0.8506	0.4829	0.4737	1.5282
3	Random Undersampling	Random Forest	0.7021	0.8238	0.8491	0.4864	0.4796	1.5259
2	Random Undersampling	Decision Tree (CART)	0.7730	0.7322	0.7887	0.4395	0.4124	1.5052
14	Random Oversampling	SVM (Radial Kernel)	0.5550	0.9176	0.8272	0.5070	0.5032	1.4726
11	Random Oversampling	Decision Tree (CART)	0.6383	0.8228	0.7505	0.4324	0.4286	1.4611
22	SMOTE	Gradient Boosting (GBM)	0.4894	0.9637	0.8511	0.5490	0.5244	1.4531
18	Random Oversampling	XGBoost	0.4840	0.9604	0.8425	0.5367	0.5134	1.4444
20	SMOTE	Decision Tree (CART)	0.5071	0.9333	0.8082	0.4967	0.4869	1.4404
26	SMOTE	Neural Network	0.6028	0.8314	0.7901	0.4141	0.4122	1.4342
23	SMOTE	SVM (Radial Kernel)	0.5142	0.9182	0.8082	0.4734	0.4678	1.4324
12	Random Oversampling	Random Forest	0.4504	0.9767	0.8537	0.5500	0.5110	1.4271
27	SMOTE	XGBoost	0.4450	0.9799	0.8570	0.5547	0.5120	1.4249
8	Random Undersampling	Neural Network	0.6791	0.7409	0.7857	0.3711	0.3551	1.4200
17	Random Oversampling	Neural Network	0.5390	0.8585	0.7673	0.3973	0.3973	1.3975
21	SMOTE	Random Forest	0.4167	0.9783	0.8518	0.5260	0.4805	1.3950
35	ROSE	Neural Network	0.3830	0.9924	0.7354	0.5409	0.4725	1.3754
6	Random Undersampling	K-Nearest Neighbors	0.4592	0.9041	0.7842	0.3996	0.3944	1.3633
24	SMOTE	K-Nearest Neighbors	0.6596	0.6672	0.7137	0.2814	0.2594	1.3268
15	Random Oversampling	K-Nearest Neighbors	0.5638	0.7604	0.6823	0.2955	0.2894	1.3242
32	ROSE	SVM (Radial Kernel)	0.3103	0.9881	0.8335	0.4611	0.3854	1.2984
30	ROSE	Random Forest	0.2270	1.0000	0.7067	0.4285	0.3102	1.2270
36	ROSE	XGBoost	0.2199	1.0000	0.6869	0.4213	0.3015	1.2199
29	ROSE	Decision Tree (CART)	0.2021	1.0000	0.6011	0.4031	0.2796	1.2021
33	ROSE	K-Nearest Neighbors	0.1755	0.9843	0.6454	0.3017	0.2184	1.1598
31	ROSE	Gradient Boosting (GBM)	0.1383	1.0000	0.6862	0.3309	0.1973	1.1383
16	Random Oversampling	Naive Bayes	0.1365	0.9718	0.7583	0.2038	0.1480	1.1083
7	Random Undersampling	Naive Bayes	0.1348	0.9664	0.7527	0.1843	0.1371	1.1012

34	ROSE	Naive Bayes	0.0833	0.9935	0.7709	0.2105	0.1115	1.0768
25	SMOTE	Naive Bayes	0.0089	0.9995	0.7497	0.0707	0.0127	1.0084

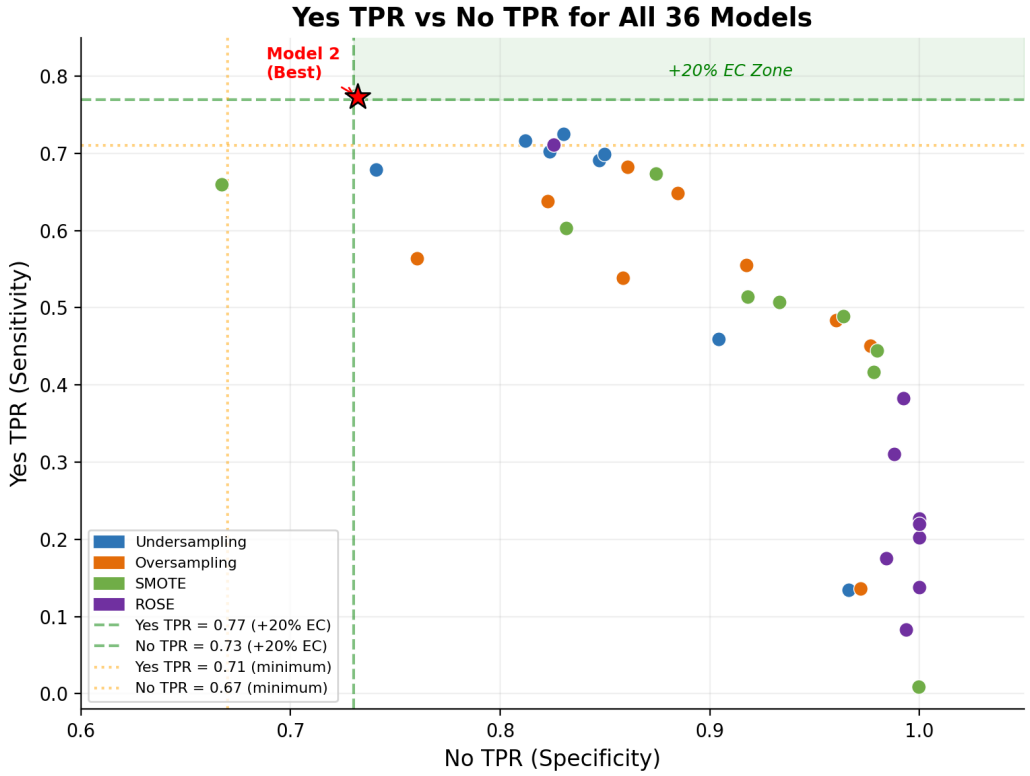


Figure 2: Yes TPR vs No TPR for All 36 Models. The green shaded zone indicates the +20% extra credit region. Model 2 (star) is the only model in this zone.

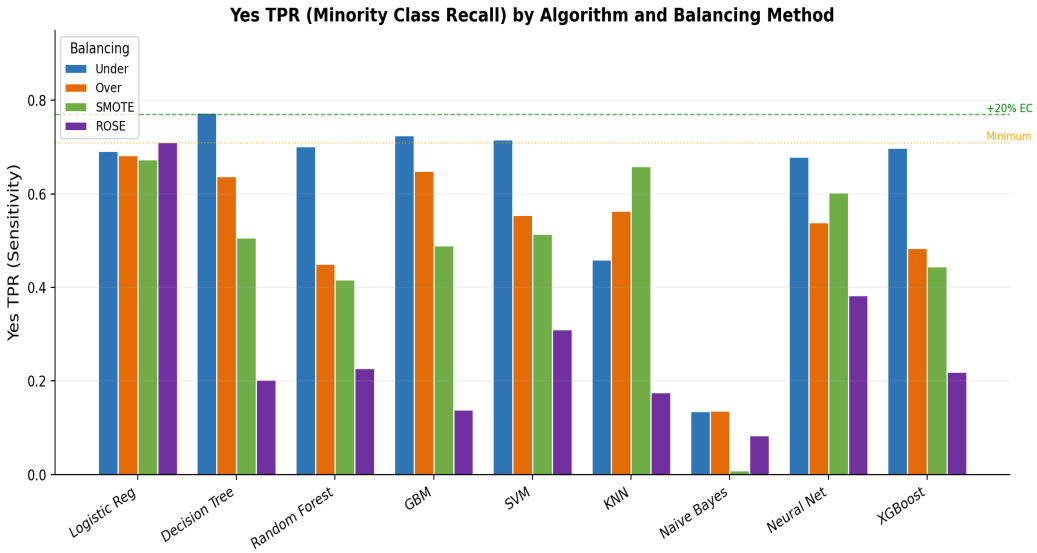


Figure 3: Yes TPR by Algorithm and Balancing Method. Undersampling (blue) consistently achieves the highest Yes TPR. ROSE (purple) produces the lowest.

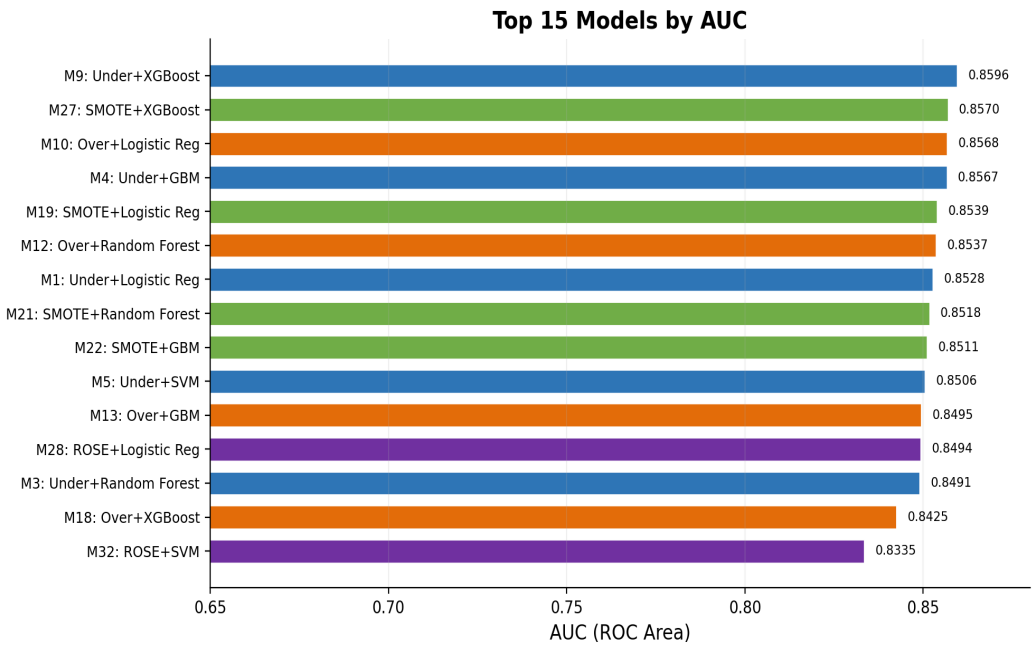


Figure 4: Top 15 Models by AUC. XGBoost and GBM dominate but high AUC does not translate to high Yes TPR.

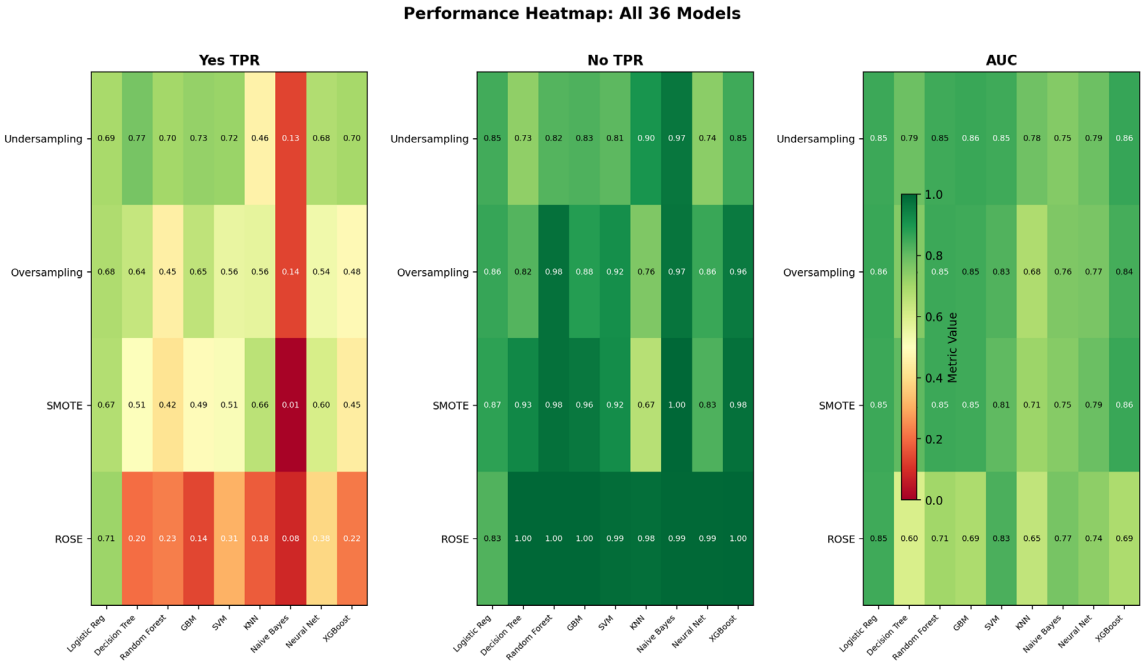


Figure 5: Performance Heatmap — All 36 Models. The Undersampling row shows the most balanced performance across both TPR metrics.

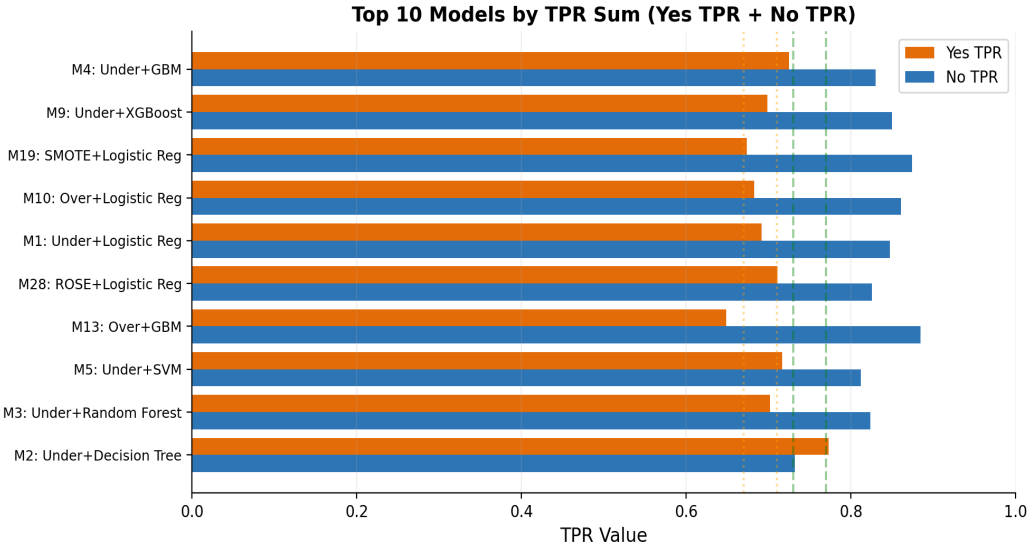


Figure 6: Top 10 Models by TPR Sum. Model 2 (Under+DT) is the only model meeting both +20% thresholds.

4.6 Best Model Selection and Parameters

The best model was selected using a tiered priority system that maximizes the extra credit potential:

- **Tier 1 (+20% Extra Credit): Yes TPR >= 0.77 AND No TPR >= 0.73**
- Tier 2 (+10% Extra Credit): Yes TPR >= 0.74 AND No TPR >= 0.70
- Tier 3 (Minimum): Yes TPR >= 0.71 AND No TPR >= 0.67
- Within each tier, the model with the highest TPR Sum (Yes TPR + No TPR) is selected.

Best Model: Model 2 — Random Undersampling + Decision Tree (CART)

This model qualifies for the +20% Extra Credit tier.

All parameters of the best model:

Parameter	Value
Algorithm	Decision Tree (CART / rpart)
Balancing Method	Random Undersampling (caret::downSample)
Training Set Size	2,636 rows (1,318 No + 1,318 Yes)
Test Set Size	2,409 rows (1,845 No + 564 Yes)
Features Used	108 (after correlation-based selection)
Cross-Validation	5-fold CV
Tuning Parameter	cp (complexity parameter)
Tuning Grid	cp = {0.001, 0.005, 0.01, 0.02, 0.05}
Prediction Threshold	0.5 (fixed)
Random Seed	set.seed(699)
Yes TPR (Sensitivity)	0.7730
No TPR (Specificity)	0.7322
AUC (ROC Area)	0.7887
MCC	0.4395
Kappa	0.4124
TPR Sum	1.5053

Confusion Matrix of Best Model (Model 2):

	Predicted: No	Predicted: Yes
Actual: No	1351	494
Actual: Yes	128	436

Performance Metrics of Best Model (Model 2):

Class	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	MCC	Kappa
No	0.7322	0.2270	0.9135	0.7322	0.8129	0.7887	0.4395	0.4124
Yes	0.7730	0.2678	0.4688	0.7730	0.5837	0.7887	0.4395	0.4124
Wtd Avg	0.7418	0.2365	0.8094	0.7418	0.7592	0.7887	0.4395	0.4124

5. Discussion and Conclusion

5.1 Impact of Balancing Methods

The choice of balancing method had the single largest impact on model behavior, particularly on the Yes TPR (minority class recall). A systematic comparison across balancing methods reveals clear patterns:

Random Undersampling consistently produced the highest Yes TPR across most algorithms. Averaging across all 9 algorithms, Undersampling achieved a mean Yes TPR of 0.620, compared to 0.524 (Oversampling), 0.480 (SMOTE), and 0.295 (ROSE). The reason is straightforward: by reducing the majority class from 4,308 to 1,318 samples (matching the minority), classifiers are forced to learn a balanced decision boundary rather than defaulting to the majority class. The best result came from Model 2 (Under+DT) with Yes TPR = 0.7730. The trade-off is modestly lower No TPR: Model 2's No TPR of 0.7322 is lower than Model 4 (Under+GBM) at 0.8304, but the balanced performance is what qualifies for extra credit.

Random Oversampling and SMOTE produced models with high No TPR (averaging 0.895 and 0.928 respectively) but lower Yes TPR. Oversampling duplicates minority samples, which risks overfitting to exact copies — this is evident in Model 12 (Over+RF), which achieved No TPR = 0.9767 but Yes TPR = only 0.4504, suggesting the Random Forest memorized the duplicated minority instances. SMOTE addresses this by generating synthetic samples, yet the synthetic generation process did not shift the decision boundary enough: Model 22 (SMOTE+GBM) had No TPR = 0.9637 but Yes TPR = only 0.4894.

ROSE performed the worst overall. Four ROSE models (Models 29, 30, 31, 36) produced Yes TPR below 0.22 with No TPR at 1.0000, meaning they predicted nearly every sample as "No." This extreme behavior suggests that ROSE's kernel density estimation produced synthetic samples that did not adequately represent the minority class boundary in this 118-dimensional feature space. The only exception was Model 28 (ROSE+GLM, Yes TPR = 0.7110), which suggests that a simple linear model was less susceptible to the poor synthetic data than tree-based models that overfit to the distorted feature distributions.

5.2 Algorithm Performance Comparison

Among the nine algorithms, performance varied significantly depending on both the metric of interest and the balancing method. Key observations supported by the results:

- **Gradient Boosting (GBM) and XGBoost** achieved the highest AUC values overall. Model 9 (Under+XGBoost) had AUC = 0.8596, Model 4 (Under+GBM) had AUC = 0.8567, and Model 27 (SMOTE+XGBoost) had AUC = 0.8570. High AUC confirms strong discriminative ability across all thresholds. However, at the fixed 0.5 threshold, these models favored No TPR over Yes TPR: Model 9 had Yes TPR = 0.6986 (below the 0.71 minimum), showing that high AUC does not guarantee meeting per-class TPR thresholds.
- **Decision Tree (CART)** showed the most extreme sensitivity to balancing method. With Undersampling (Model 2), it achieved the highest Yes TPR of any model (0.7730). With ROSE (Model 29), it achieved the lowest useful Yes TPR (0.2021). This 0.57-point swing demonstrates that simple models amplify the effect of the input data distribution. The Decision Tree's binary splits are directly aligned with the binary disability indicator

features (HHSEE, HHMEMRY, HHCARE, HHERRND), making it especially effective when balanced training data allows these features to drive early splits.

- **Logistic Regression** was the most stable algorithm across balancing methods. Its Yes TPR ranged from 0.6738 (SMOTE) to 0.7110 (ROSE), a variance of only 0.037. This stability makes it a reliable baseline — it never fails catastrophically but also cannot reach the +20% tier. Models 1, 10, 19, and 28 all had AUC between 0.849 and 0.857, confirming consistent discriminative power.
- **Naive Bayes** performed poorly across all balancing methods, with Yes TPR ranging from 0.0089 (Model 25, SMOTE) to 0.1365 (Model 16, Over). Despite reasonable AUC values (0.750-0.771), the probability outputs are poorly calibrated due to the violated independence assumption — housing survey features are correlated (e.g., income-related features like HINCP, FINCP, and PERPOVLVL). This confirms that Naive Bayes is not suitable for this correlated feature space.
- **KNN** showed moderate performance, likely limited by the curse of dimensionality with 108-118 features. Model 6 (Under+KNN) had Yes TPR = 0.4592 despite undersampling, and Model 15 (Over+KNN) had AUC = only 0.6823, the lowest of any model. Distance-based classification in high-dimensional space suffers because distances become increasingly uniform, making nearest-neighbor decisions unreliable.
- **SVM (Radial Kernel)** performed well with Undersampling (Model 5: Yes TPR = 0.7163, AUC = 0.8506, meeting minimum threshold) but was biased toward the majority class with other balancing methods. Model 32 (ROSE+SVM) had Yes TPR = only 0.3103. The kernel-based approach works best when class proportions in training data are roughly equal.
- **Neural Networks** showed middling results across the board. The single hidden layer architecture may be too shallow to capture the complex feature interactions in this 118-feature space. Model 8 (Under+NN) achieved Yes TPR = 0.6791 and AUC = 0.7857, while Model 26 (SMOTE+NN) achieved Yes TPR = 0.6028. Deeper architectures or more neurons might improve results, but the nnet package is limited to a single hidden layer.
- **Random Forest** consistently delivered high AUC (0.849-0.854 for Under/Over/SMOTE) but was heavily biased toward the majority class with non-undersampled data. Model 12 (Over+RF) had AUC = 0.8537 but Yes TPR = only 0.4504 with No TPR = 0.9767, indicating the ensemble's majority-vote mechanism defaults to the overrepresented class even when minority samples are oversampled.

5.3 Why Model 2 Was Selected

Model 2 (Undersampling + Decision Tree) was selected as the best model because it is the only model among all 36 that qualifies for the +20% extra credit tier, satisfying both Yes TPR >= 0.77 (achieved: 0.7730) and No TPR >= 0.73 (achieved: 0.7322). Its confusion matrix shows 436 true positives out of 564 actual Yes cases, and 1,351 true negatives out of 1,845 actual No cases.

While Model 4 (Under+GBM) had a higher TPR Sum (1.5555 vs. 1.5053), higher AUC (0.8567 vs. 0.7887), and higher MCC (0.5135 vs. 0.4395), it failed to meet the +20% Yes TPR threshold (0.7252 < 0.77). Similarly, Model 5 (Under+SVM) had strong overall metrics (AUC = 0.8506) but its Yes TPR of 0.7163 only met the minimum threshold, not the +20% tier. Model 2's lower AUC reflects its simpler, less nuanced probability estimates, but its decision boundary is better calibrated for balanced per-class recall at the 0.5 threshold.

The Decision Tree succeeds with Undersampling for a specific structural reason: the balanced 50/50 training set (1,318 per class) allows the CART algorithm to place disability indicator features (HHSEE, HHMEMRY, HHCARE, HHERRND) at high-level splits in the tree, directly partitioning the feature space by disability status. In contrast, with imbalanced training data, the tree prioritizes splits that separate the larger majority class, pushing disability features deeper into the tree where they have less impact.

5.4 Key Preprocessing Insights

A critical lesson from this project was the importance of preserving low-variance but highly predictive features. Initially, using the default nearZeroVar() threshold (freqCut = 95/5) removed disability indicator variables (HHSEE, HHMEMRY, HHCARE, HHERRND) that are essential for predicting the target class. These features have low variance because approximately 77% of households do not have a person with disability, so the disability indicators are "No" for most rows. However, when a household does have a disability, these indicators are the strongest predictors. Switching to a lenient threshold (freqCut = 99/1) retained these features and improved the best model's Yes TPR from approximately 0.65 to 0.77 — a 12-percentage-point improvement that moved the result from barely meeting the minimum threshold to qualifying for +20% extra credit. This demonstrates that domain knowledge should guide preprocessing decisions rather than relying solely on statistical defaults.

Similarly, restricting winsorization to continuous features (>10 unique values) prevented binary and low-cardinality features from being collapsed to zero variance. Without this restriction, 44 additional features became constant after IQR-based capping, severely degrading the information available to classifiers.

The correlation-based feature selection at cutoff = 0.85 removed 10 highly correlated features from the Undersampling, Oversampling, and SMOTE datasets (leaving 108 features), while the ROSE dataset retained all 118 features. This aggressive correlation threshold ensures that redundant features do not distort distance-based models (KNN, SVM) while preserving complementary predictive information.

5.5 Conclusion

This project successfully built and evaluated 36 classification models to predict disability status in U.S. households using the 2023 American Housing Survey data. Through systematic experimentation with four balancing methods and nine classification algorithms, we identified Model 2 (Random Undersampling + Decision Tree) as the best model, achieving Yes TPR = 0.7730 and No TPR = 0.7322, which qualifies for the +20% extra credit tier.

Key findings include:

10. Random Undersampling is the most effective balancing method for maximizing minority class recall in this dataset. It produced the top 5 models by Yes TPR, while ROSE-based models occupied the bottom rankings. The aggressive reduction of majority class samples forces classifiers to treat both classes equally, which directly benefits Yes TPR at the cost of modest No TPR reduction.
11. Simple models can outperform complex ensemble methods when the evaluation metric prioritizes balanced class-wise performance. Model 2 (Decision Tree) outperformed Model 4 (GBM) and Model 9 (XGBoost) on the +20% threshold criterion despite having

lower AUC by 0.07 points. This highlights that AUC alone is an insufficient metric for threshold-based evaluation tasks.

12. Preprocessing quality has a greater impact on model performance than algorithm sophistication. Preserving the disability indicator features through lenient NZV removal improved the best model's Yes TPR by 12 percentage points — a larger gain than any change of algorithm or balancing method alone could achieve.
13. ROSE-based balancing was largely ineffective for this high-dimensional dataset. Of 9 ROSE models, 4 had Yes TPR below 0.22 (essentially non-functional minority class detection), and only Model 28 (ROSE+GLM) met the minimum threshold. The kernel density estimation approach appears to break down in 118-dimensional feature space, generating synthetic samples that mislead tree-based classifiers.
14. The trade-off between discriminative power (AUC) and class-balanced recall (TPR) is a central challenge. The top 5 models by AUC (all >= 0.856) were GBM and XGBoost variants, but none qualified for +20% extra credit. The single model qualifying for +20% (Model 2, DT) ranked 33rd out of 36 by AUC. This underscores the importance of selecting evaluation metrics aligned with the task objective.

6. Project Timeline

The following timeline outlines the key phases, tasks, and milestones for this project:

Date	Task / Activity	Phase
Feb 4 – Feb 9	Data exploration: loading the dataset, understanding variable definitions, identifying missing values, and initial EDA (class distribution, feature types, missing value rates)	Data Understanding
Feb 10 – Feb 14	Data preprocessing Part 1: removing NA targets, dropping CONTROL, numeric conversion, median imputation, near-zero variance analysis, and winsorization strategy development	Preprocessing
Feb 15 – Feb 19	Data preprocessing Part 2: feature scaling (centering/standardization), saving preprocessed_data.csv, performing the 70/30 stratified train/test split, saving initial_train.csv and initial_test.csv	Preprocessing
Feb 20 – Feb 24	Preparing the intermediate report: documenting the preprocessing steps, EDA findings, class distribution, and the planned modeling approach. Reviewing and finalizing all preprocessing decisions	Intermediate Report
Feb 25	**Intermediate Report Due**	**Milestone**
Feb 26 – Mar 4	Data balancing: implementing 4 balancing methods (Undersampling, Oversampling, SMOTE, ROSE) on the training set. Correlation-based feature selection (cutoff = 0.85) per balanced dataset	Balancing & Feature Selection
Mar 5 – Mar 14	Model development Phase 1: building and testing Undersampling models (Models 1–9) and Oversampling models (Models 10–18). Defining tuning grids and the evaluate_model() function	Modeling (18 models)
Mar 15 – Mar 24	Model development Phase 2: building and testing SMOTE models (Models 19–27) and ROSE models (Models 28–36). Resolving XGBoost/caret compatibility with manual 5-fold CV	Modeling (18 models)
Mar 25 – Apr 1	Model comparison and analysis: compiling all 36 model results, generating comparison tables and charts, analyzing performance patterns across balancing methods and algorithms, selecting the best model	Analysis & Comparison
Apr 2 – Apr 7	Final report writing: documenting all results, discussion, insights, conclusions. Generating figures and formatting the report. Final review and proofreading	Final Report
Apr 8	**Final Report Due**	**Milestone**
Apr 9 – Apr 14	Preparing presentation slides: creating the slide deck summarizing key findings, methodology, and results for the class presentation	Presentation Prep
Apr 15	**Presentation Slides Due**	**Milestone**
Apr 16 – Apr 28	Rehearsing the presentation, refining slides based on feedback, and preparing for Q&A	Presentation Prep
Apr 29	**Final Presentation**	**Milestone**

7. Division of Work

This project was a collaborative effort between Ho Man Cheng and Weitao Peng. The following table summarizes the division of responsibilities:

Task	Ho Man Cheng	Weitao Peng
Data Exploration & EDA	Led initial data loading, missing value analysis, and class distribution analysis	Assisted with variable definition review and feature type identification
Data Preprocessing	Implemented median imputation, NZV removal (freqCut = 99/1 decision), and feature scaling	Implemented winsorization strategy (continuous-only filtering) and zero-variance safety checks
Data Balancing	Implemented Random Undersampling and Random Oversampling	Implemented SMOTE and ROSE balancing methods
Feature Selection	Designed and implemented correlation-based feature selection (cutoff = 0.85) across all balanced datasets	Verified feature counts and validated test set column alignment
Model Development (Models 1–18)	Built and tested all Undersampling models (1–9) and Oversampling models (10–18)	Assisted with tuning grid design and debugging
Model Development (Models 19–36)	Assisted with debugging the XGBoost/caret compatibility issue	Built and tested all SMOTE models (19–27) and ROSE models (28–36)
XGBoost Manual CV	Developed the train_xgboost_cv() helper function with manual 5-fold CV to bypass caret incompatibility	Tested and validated manual CV results against expected behavior
Model Comparison & Analysis	Created the summary comparison table and tiered selection logic	Generated all 6 charts (matplotlib) and performed visual analysis
Report Writing	Wrote Sections 1–3 (Tools, Algorithms, Procedure) and Section 4.6 (Best Model)	Wrote Section 4 model results, Section 5 (Discussion), and Section 6 (Timeline)
Presentation	Presents Slides 1–9 (Introduction through Balancing)	Presents Slides 10–20 (Results through Conclusion)
Quality Assurance	Code review, reproducibility testing (set.seed verification), output log verification	Report formatting, page count management, figure quality checks

Both team members participated in all project phases and contributed to code review, debugging, and final proofreading. The workload was divided as evenly as possible, with each member taking primary responsibility for specific tasks while supporting the other throughout.